

Sprint 2 Report

Team 3:

XXXXXXXX XXXXXX

XXXX XXXXXX

XXXXX XXXXXX

Julien Chagnon

XXXXXXXXXXXXXXXXXX

XXXXXXXXXXXX XXXXX

11/01/2025

1. Project Overview

This project is an academic scheduling tool developed in C++ with a Qt GUI, designed for Queen's University students. It imports .ics course schedules from SOLUS and combines them with user-entered tasks to automatically generate a personalized, conflict-free study calendar. Unlike existing static tools such as Google Calendar or MyStudyLife, this system adapts in real time to new deadlines and priorities using a weighted scheduling algorithm. Its success is defined by being able to generate schedules within 3 seconds and achieving a user satisfaction rating of at least 8/10.

2. Sprint Backlog

The sprint backlog is outlined below. The automated scheduling feature remains in progress and will be fully implemented in Sprint 3 under xxxxxxxxx and Julien.

User story (one liner + estimate)	Priority	Assigned to	Status (completed/not)
As a student, I want to view my tasks in a list view so that I can clearly see what is coming up. (4 hours)	High	xxxxx	Completed
As a student, I want to be able to add, edit and delete tasks so that I can adjust for changing tasks. (4 hours each)	High	xxxxx (Front end), xxxxxxxxx (Back end)	Completed
As a student, I want to be able to add events to my schedule so that I can account for scheduled activities. (4 hours)	High	xxxxxxx	Completed
As a student, I want my tasks to be added to my calendar in a free slot so that I know how to split up my time. (8 hours)	High	xxxxxxx	Completed
As a student, I want a welcome page that clearly explains the setup and the first step I need to take. (4 hours)	Medium	xxxxxxx	Completed

3. GitLab Activity

The team used GitLab for Sprint 2 to coordinate work and integrate new features. Each member contributed through regular commits and merges tied to an assigned task from the user stories. Branches were used for components such as task management, event creation, and the UI layout, which were then merged into the main branch once testing was complete. The group collaborated by pulling each other’s changes, resolving merge conflicts together, and verifying builds on both Mac and Windows. The commit and merge history shows steady contributions across the sprint, demonstrating effective teamwork and continuous progress.

Each member contributed actively, as shown in the screenshots below. xxxxx focused on the task list and UI polish, xxxxxxxxx implemented the backend task logic, xxxxxxxx handled event creation, Julien supported task-calendar integration, xxxxxxxx worked on interface layout and testing, and xxxx verified features and documented changes.

Oct 27, 2025  added edit and delete buttons to the task xxxxx authored 5 days ago  Resolved merge conflicts in .gitignore and main_window.cpp xxxxx authored 6 days ago  Add TaskListPanel and AddTaskDialog UI components and updated main window xxxxx authored 6 days ago  Add .gitignore to clean up Qt build files xxxxx authored 6 days ago	Nov 01, 2025  Add welcome dialog implementation xxxxx authored 1 day ago  Add welcome dialog implementation xxxxx authored 1 day ago
Oct 20, 2025  Fixing Calendar columns and build files JulienChagnon authored 1 week ago  Calendar column fix, Removing Build Files JulienChagnon authored 1 week ago  Merge branch 'revert-bc4dd21a' into 'main' *** Julien Chagnon (22PCLT) authored 1 week ago  Revert "Fixed Calendar input window" *** Julien Chagnon (22PCLT) authored 1 week ago  Fixed Calendar input window JulienChagnon authored 1 week ago  Fixed calendar input window JulienChagnon authored 1 week ago	Oct 31, 2025  Merged visual blocks for same events on calendar. Allowed for tasks of 30min... *** JulienChagnon authored 2 days ago  Merge branch 'main' into Julien JulienChagnon authored 2 days ago  Merge branch 'Julien' of... *** JulienChagnon authored 2 days ago
	Oct 30, 2025  Merged the main branch with changes so user can manually add an event xxxxx authored 3 days ago  Fixed some of the task stuff, still errors when overloading or switching weeks xxxxx authored 3 days ago
	Oct 29, 2025  I implemented the whole data struct for tasks, 30-min scheduling that fills... *** xxxxx authored 3 days ago  Add setup instructions file for Mac and PC xxxxx authored 4 days ago  Ignore .DS_Store globally xxxxx authored 4 days ago  Cleaned up files, moved into folders, updated everything this has the tasks... *** xxxxx authored 4 days ago

Nov 02, 2025



Fix to main_window.h

xxxx

authored 1 hour ago



Fix to main_window.h to allow the welcome page to upload file

xxxx

authored 1 hour ago



Main file updated to fix welcome page file uploading

xxxx

authored 1 hour ago



Fixing calendar view cutoff

xxxx

authored 1 hour ago



Header file for welcome message

xxxx

authored 2 hours ago

Completed
(closed)

2

List view for all tasks

#14

Functional Requirement



Done

User manual task entry and editing

#4

Functional Requirement



Done

4. Team Meetings

The team met twice weekly throughout Sprint 2, during tutorials and outside class up to the deadline. The meetings below describe what was done during each, and the last two meetings October 27 and 30 were used to write the report.

4.1 Meeting Logs

Meeting 1 – Mon Oct 6th (10:30 AM Tutorial)

Attendees: xxxxxxxx xxxxxxx, xxxxx xxxxx, Julien Chagnon, xxxx xxxxxxx, xxxxxxxxxxx xxxxx, xxxxxxxx xxxxxxx

Topics discussed:

- Reviewed sprint 1 progress and defined Sprint 2 goals
- Planned implementation of manual task entry and list view display
- Discussed integration of task data with calendar model for future functional requirement.

Decisions made:

- Assign xxxxx to task creation and list view implementation
- Assign xxxxxxx to add event creation logic

Next steps:

- Create task card UI templates and define data structures for Task and Event classes

Meeting 2 – Thu Oct 9th (10:30 AM Outside Class)

Attendees: xxxxxxxx xxxxxxx, xxxxx xxxxx, Julien Chagnon, xxxx xxxxxxx, xxxxxxxxxxx xxxxx, xxxxxxxx xxxxxxx

Topics discussed:

- Progress check on task add/edit features
- Bug reports with Qt layouts

Decisions made:

- Switch to nested vertical layouts for task cards
- Adopt consistent style for buttons and task labels

-Continue testing on both Windows and Mac to catch build issues

Next steps:

-xxxxxxx and xxxx to finalize UI alignment and styling

Meeting 3 – Mon Oct 20th (10:30 AM Tutorial)

Attendees: xxxxxxxx xxxxxxx, xxxxx xxxxxx, Julien Chagnon, xxxx xxxxxxx, xxxxxxxxxxx xxxxx,
xxxxxxxx xxxxxxx

Topics discussed:

- Demo of completed task list view
- Integration of events and tasks into data structure
- UI polish

Decisions made:

- Use centralized TaskManager object to store tasks and emit update signals to views
- Document code and update GitLab milestones for completed stories

Next steps:

- Begin implementing placement of tasks into calendar time slots
- Managing tasks of differing lengths (incorporating 30min increments)

Meeting 4 – Thu Oct 23rd (10:30 AM Outside Class)

Attendees: xxxxxxxx xxxxxxx, xxxxx xxxxxx, Julien Chagnon, xxxx xxxxxxx, xxxxxxxxxxx xxxxx,
xxxxxxxx xxxxxxx

Topics discussed:

- Reviewed remaining bugs and incomplete user stories
- Discussed allocation of task study time

Decisions made:

- Opt for drop down menu for time selection rather than typed entry

Next steps:

- Implement study time allocation algorithm and add to visual interface

5. Completed Sprint Evidence

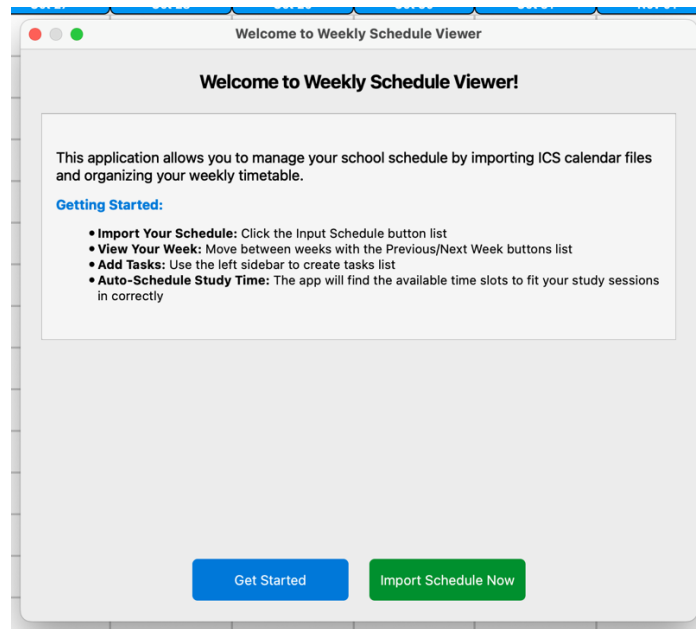
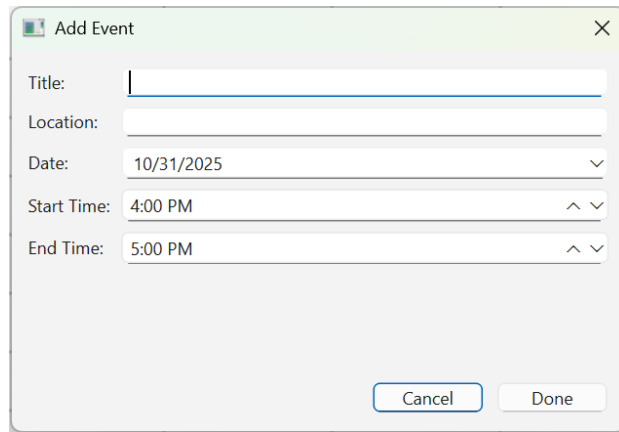


Figure 1: Entry window with initial prompts

A screenshot of a macOS-style window titled "Add Task". The window has a light gray background and a white title bar with red, yellow, and green window control buttons. The main content area has a heading "Add Task" followed by four form fields: "Name" with a text input field containing the placeholder "Enter task name", "Due Date" with a date picker showing "Oct 29, 2025", "Estimated Effort" with a spinner box showing "1 hrs", and "Priority" with a dropdown menu showing "Select priority". At the bottom right of the window are two buttons: a white "Cancel" button and a pink "OK" button.

Figure 2: Task info entry window



The 'Add Event' dialog box is a light gray window with a green title bar containing a small icon and the text 'Add Event'. It features a close button (X) in the top right corner. The form contains five input fields: 'Title' (empty), 'Location' (empty), 'Date' (set to '10/31/2025'), 'Start Time' (set to '4:00 PM'), and 'End Time' (set to '5:00 PM'). Each of the last three fields has a small downward arrow on its right side. At the bottom right, there are two buttons: 'Cancel' and 'Done'.

Figure 3: Event info entry window

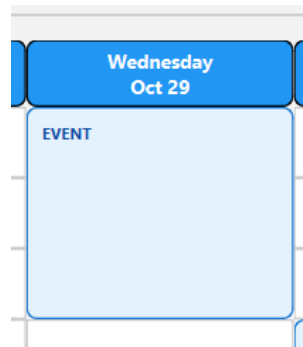


Figure 4: Custom event block in calendar with title "EVENT"

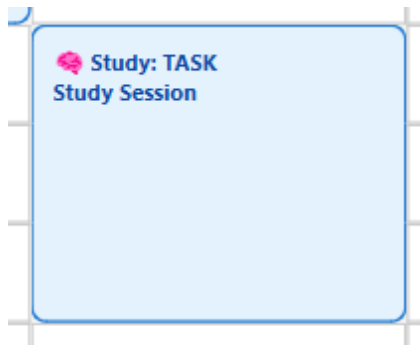


Figure 5: Allocated study/work time for sample task

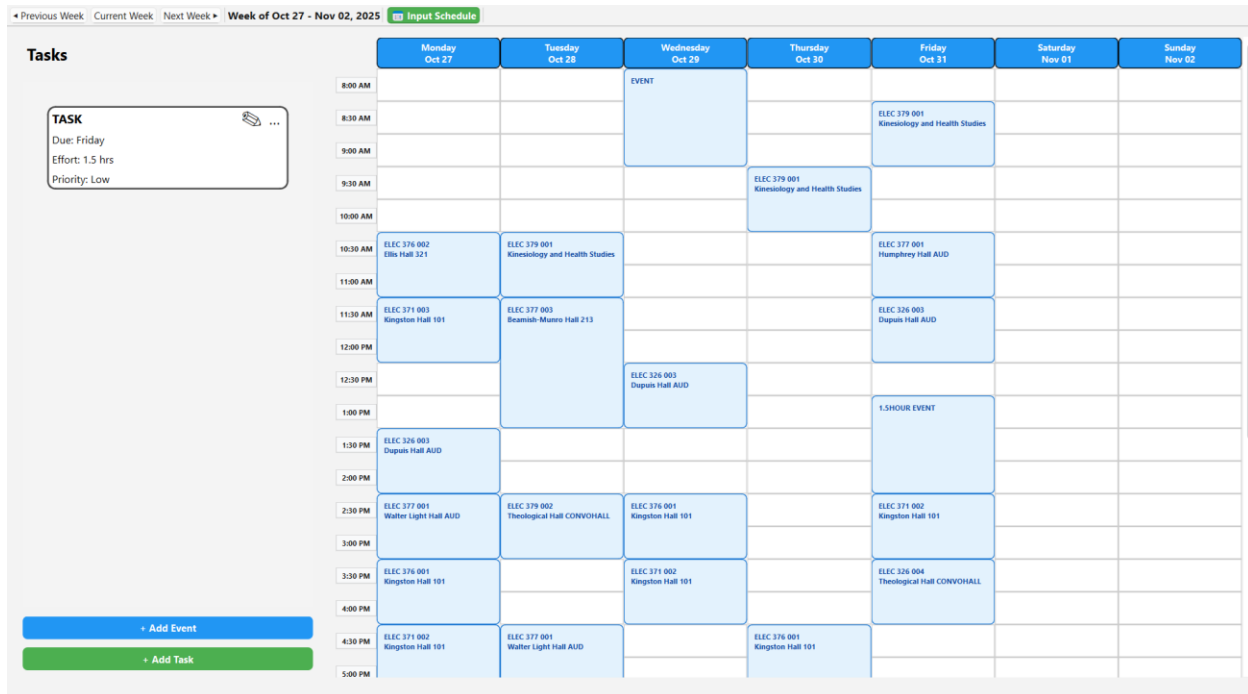


Figure 6: Current view of main calendar window, including vertical list view of tasks

6. Sprint Review

Our TA meeting is scheduled for Monday October 3rd at 6:30pm.

7. Sprint Review Details

Our sprint review was held during the tutorial on Monday, October 28, attended by xxxxx xxxxx, xxxxxxxx xxxxxx, Julien Chagnon, xxxxxxxx xxxxxxxx, and xxxxxxxxxx xxxxx. During the meeting, the team demonstrated the completed features including the task list view, task addition, editing and deletion functionality and the addition of events.

The team successfully met the sprint goal. All the high priority features planned for this sprint were completed. The “Add Task” button was created with the functionality to add tasks with a name, due date, time commitment and priority. The list view for tasks was implemented and tested. The edit and delete functionality for tasks was finished. Event addition was also completed and verified to display correctly. These new features will allow users to both visualize their weekly academic schedules and everything that they must get done.

One challenge was ensuring that the edit and delete buttons for each task were positioned and styled consistently within each task card. We experimented with different layout combinations (QHBoxLayout, QVBoxLayout, and nested wrappers) to prevent clipping and horizontal scrolling. We also tested multiple icon options and UI adjustments to make the task cards visually consistent, without relying on external image imports or dependencies (like QtAwesome) that introduced build errors.

After reviewing the outcomes of this sprint, the team updated the product backlog. The next sprint will focus on updating the schedule to account for new tasks being added and more study time being allocated. The user story, “As a student, I want the system to automatically update my schedule when new projects and tasks come up so that I don’t have to constantly re-schedule” was added to the backlog.

During our first sprint review, our TA suggested exploring ways to incorporate assignment deadlines and due dates directly from OnQ into the schedule. While this would improve usability, the team determined that it is currently outside of the project’s scope, as direct API access to OnQ would be required to import data automatically. Instead, during this sprint, we implemented manual task creation allowing students to input their own assignment deadlines. This approach maintains flexibility while avoiding the need for integration with external systems.

8. Sprint Retrospective

Date: October 30th, 2025

Time: 10:30 AM (Thursday meeting)

Attendees: XXXXXXXX XXXXXX, XXXX XXXXXX, XXXXX XXXXX, Julien Chagnon, XXXXXXXX XXXXXXXX, XXXXXXXXXXX XXXXX

Scrum Master: XXXXXXXX XXXXXXXX

Product Owner: XXXXX XXXXX

Sprint 2 focused on refining the UI and adding more manual entry features across the application. The team successfully completed additional UI features and general refinement. This included the ability to add, delete, edit, and view tasks, as well as add custom events to view on the calendar. The GitLab was well kept, the user stories for this sprint were well defined and proper subtasks and milestones were created. The team also became much more familiar with the software being used making it easier to implement changes. This helped with project progress when front end and backend were being completed by different team members and needed to be integrated.

The team faced challenges with the build and integration of the application across different devices. Team members lost time as a lot of problems arose when rebuilding the project after others pushed changes because of platform differences like windows vs mac. The UI was improved since sprint 1 as planned however the team will continue to update and polish the UI as there is more room for user friendliness and readability. The team also planned to implement the automated study schedule however it hasn't been fully completed and remains in the backlog for Sprint 3.

During team meetings, we reviewed progress, completed work, and defined goals for upcoming functionality and features we wanted to implement. The team accomplished this by assigning tasks for that week and discussing the general next steps for the project. The team updated GitLab milestones and user stories throughout the sprint to reflect their progress. The team also communicated regularly through Teams and message chats. Overall, the communication both in and outside of meetings was very effective throughout the sprint.

For Sprint 3, the focus will be on implementing the remaining functional and non-functional requirements outlined in the user stories. This includes developing the study schedule generator, which will automatically update as a user adds new tasks and events to the calendar. The team also plans to implement better security for accessing the application above just hosting the project locally, this will most likely be done through a sign in page. Because sprint 3 will be the final sprint of the project more emphasis will be put on testing the application and its corner cases before final submission.